

QUICK REFERENCE GUIDE Capstone Project - CSC 489 Spring 2026

PURPOSE

This guide provides quick setup instructions, common commands, and troubleshooting solutions for your capstone project.

Keep this handy while working on your project!

TABLE OF CONTENTS

1. Quick Setup with Virtual Environment
 2. Common Commands
 3. File Structure Reference
 4. API Testing Tips
 5. Troubleshooting Common Issues
 6. Git/GitHub Tips
 7. Testing with ZAP Proxy
 8. Useful Tools
 9. Before Submission Checklist
-
-

1. QUICK SETUP WITH VIRTUAL ENVIRONMENT

FIRST TIME SETUP:

1. Extract the starter template ZIP file
2. Setup Backend with Virtual Environment:

macOS/Linux:

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Windows:

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

✅ You should see (venv) at the start of your terminal prompt

3. Setup Frontend (in NEW terminal):

```
cd frontend
npm install
```

RUNNING YOUR APP (EVERY TIME):

Terminal 1 (Backend):

```
cd backend
source venv/bin/activate      # macOS/Linux
# OR
venv\Scripts\activate         # Windows
```

```
python app.py
```

Backend runs on: <http://localhost:5000>

Terminal 2 (Frontend):

```
cd frontend
npm run dev
```

Frontend runs on: <http://localhost:3000>

Open browser: <http://localhost:3000>

You should see “Backend Connected” ✅

STOPPING YOUR APP:

Terminal 1: Press Ctrl+C, then type: deactivate Terminal 2: Press Ctrl+C

2. COMMON COMMANDS

BACKEND (Flask):

Activate virtual environment:

```
source venv/bin/activate    # macOS/Linux
venv\Scripts\activate      # Windows
```

Start server (after activating venv):

```
python app.py
```

Install new package (after activating venv):

```
pip install package-name
pip freeze > requirements.txt
```

Deactivate virtual environment:

```
deactivate
```

Reset database:

```
rm database.db    # macOS/Linux
del database.db   # Windows
python app.py
```

(Database auto-recreates with test user)

Check Python version:

```
python --version
```

(Need 3.8+)

FRONTEND (Next.js):

Start dev server:

```
npm run dev
```

Install new package:

```
npm install package-name
```

Build for production:

```
npm run build
```

Clear cache and reinstall:

```
rm -rf node_modules package-lock.json # macOS/Linux
rmdir /s node_modules & del package-lock.json # Windows
npm install
```

Check Node version:

```
node --version
```

(Need v18+)

3. FILE STRUCTURE REFERENCE

YOUR PROJECT STRUCTURE:

```
YourApp/
├── backend/
│   ├── venv/           # Virtual environment (DO NOT commit!)
│   ├── app.py          # Flask application
│   ├── database.db     # SQLite database (auto-created)
│   ├── requirements.txt # Python dependencies
│   └── README.md
├── frontend/
│   ├── node_modules/   # Node packages (DO NOT commit!)
│   ├── app/
│   │   ├── page.jsx    # Home page (/)
│   │   ├── layout.jsx  # Root layout
│   │   ├── login/
│   │   │   └── page.jsx # Login page (/login)
│   │   ├── profile/
│   │   │   └── page.jsx # Profile page (/profile)
│   ├── package.json
│   └── next.config.js
├── .gitignore          # Files to exclude from git
└── README.md           # Main setup instructions
```

WHERE TO ADD YOUR CODE:

Backend: • Add API endpoints in: backend/app.py • Add database tables in: init_db() function

Frontend: • Add new pages in: frontend/app/[page-name]/page.jsx • Modify home page: frontend/app/page.jsx

WHAT NOT TO COMMIT:

- backend/venv/ (huge, regenerate with pip install)
 - frontend/node_modules/ (huge, regenerate with npm install)
 - backend/database.db (auto-creates)
 - .DS_Store, **pycache** (OS/Python junk)
-
-

4. API TESTING TIPS

TESTING BACKEND APIs (Without Frontend):

Option 1: Browser Visit: <http://localhost:5000/api/test>

Option 2: curl (Command Line)

GET request

```
curl http://localhost:5000/api/test
```

POST request

```
curl -X POST http://localhost:5000/api/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"admin123"}'
```

Option 3: Postman (GUI Tool) 1. Download: <https://www.postman.com/downloads/> 2. Create new request 3. Set URL: <http://localhost:5000/api/endpoint> 4. Set method: GET/POST 5. Add JSON body if needed 6. Click Send

Option 4: Browser DevTools 1. Open page in browser 2. Press F12 → Console tab 3. Type:

```
fetch('http://localhost:5000/api/test')  
  .then(r => r.json())  
  .then(console.log)
```

MAKING API CALLS FROM FRONTEND:

Example (Next.js):

```
'use client';  
import { useState } from 'react';  
  
export default function ExamplePage() {  
  const [data, setData] = useState(null);  
  
  const fetchData = async () => {  
    try {  
      const response = await fetch('http://localhost:5000/api/endpoint');
```

```

    const json = await response.json();
    setData(json);
  } catch (error) {
    console.error('Error:', error);
  }
};

return (
  <div>
    <button onClick={fetchData}>Fetch Data</button>
    {data && <pre>{JSON.stringify(data, null, 2)}</pre>}
  </div>
);
}

```

Remember: Use 'use client' at top of file when using hooks!

5. TROUBLESHOOTING COMMON ISSUES

PROBLEM: “externally-managed-environment” error (macOS/Linux)

ERROR: Can’t install packages with pip FIX:

```

# You forgot to create/activate venv!
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

```

Look for (venv) in your terminal prompt to confirm activation.

PROBLEM: Backend won’t start

ERROR: “Address already in use” or “Port 5000 is busy” FIX:

```

# Find process using port 5000
lsof -i :5000          # macOS/Linux
netstat -ano | findstr :5000 # Windows

```

```

# Kill the process
kill -9 [PID]          # macOS/Linux
taskkill /PID [PID] /F # Windows

```

```
# Or use different port in app.py:  
app.run(debug=True, port=5001)
```

ERROR: "ModuleNotFoundError: No module named 'flask'" FIX:

```
# Activate venv first!  
source venv/bin/activate  
pip install flask flask-cors
```

ERROR: "OperationalError: no such table: users" FIX:

```
rm database.db  
python app.py
```

Database will auto-recreate.

PROBLEM: Frontend won't start

ERROR: "Cannot find module 'next'" FIX:

```
cd frontend  
npm install
```

ERROR: "Port 3000 is already in use" FIX:

```
# Kill process on port 3000  
lsof -i :3000          # macOS/Linux  
netstat -ano | findstr :3000 # Windows  
kill -9 [PID]          # macOS/Linux  
taskkill /PID [PID] /F  # Windows
```

```
# Or use different port:  
npm run dev -- -p 3001
```

PROBLEM: CORS errors

ERROR: "Access-Control-Allow-Origin" in browser console FIX: Backend (Flask) - Make sure you have:

```
from flask_cors import CORS  
app = Flask(__name__)  
CORS(app) # This line!
```

PROBLEM: Database errors

ERROR: "database is locked" FIX:

```
# Close all connections, then:  
rm database.db  
python app.py
```

ERROR: "OperationalError: database disk image is malformed" FIX:

```
# Backup data first if needed, then:  
rm database.db  
python app.py
```

PROBLEM: Frontend can't connect to backend

ERROR: "Failed to fetch" or "Network request failed" CHECKLIST: ☐ Is Flask running?
Check Terminal 1 ☐ Is venv activated? Look for (venv) in prompt ☐ Is it on port 5000?
Check Flask output ☐ Did you add CORS? Check app.py ☐ Is URL correct?
http://localhost:5000 (not https) ☐ Check browser console for error details

TEST: Open http://localhost:5000 in browser Should see Flask response (not error)

PROBLEM: Changes not showing up

Frontend changes not appearing: • Hard refresh: Ctrl+Shift+R (Windows/Linux) or
Cmd+Shift+R (Mac) • Clear cache: DevTools → Network tab → Disable cache • Restart dev
server: Stop (Ctrl+C) and run npm run dev again

Backend changes not applying: • Restart Flask server: Stop (Ctrl+C) and run python app.py
again • Flask auto-reloads in debug mode, but not always perfectly

PROBLEM: Forgot if venv is activated

HOW TO CHECK: • Look for (venv) at start of terminal prompt • Run: which python (should
show path with venv)

IF NOT ACTIVATED:

```
cd backend  
source venv/bin/activate    # macOS/Linux  
venv\Scripts\activate      # Windows
```

6. GIT/GITHUB TIPS (Optional)

INITIAL SETUP:

```
cd your-project
git init
git add .
git commit -m "Initial commit"
```

.gitignore FILE (ALREADY INCLUDED):

The template includes .gitignore that excludes: • venv/ (virtual environment) • node_modules/ (Node packages) • *.db (database files) • **pycache/** (Python cache) • .DS_Store (macOS files)

IMPORTANT: Never commit venv/ or node_modules/!

BASIC GIT WORKFLOW:

```
# Check status
git status

# Add changes
git add .

# Commit
git commit -m "Added login feature"

# Push to GitHub (if using)
git push origin main
```

SHARING WITH TEAM:

Option 1: GitHub • Create repo on GitHub • Push code • Team clones repo • Each person creates their own venv: `cd backend python3 -m venv venv source venv/bin/activate pip install -r requirements.txt`

Option 2: ZIP file • Zip entire folder (excluding venv, node_modules) • Share via Canvas/Google Drive • Team extracts and runs setup

7. TESTING WITH ZAP PROXY

ZAP SETUP (For Week 11 - Attack Phase):

1. Download ZAP: <https://www.zaproxy.org/download/>
2. Configure Firefox Proxy: • Settings → Network Settings → Manual Proxy • HTTP Proxy: localhost • Port: 8080 • Check “Also use this proxy for HTTPS”
3. Start ZAP, click “Automated Scan”
4. Enter URL: <http://localhost:3000>
5. Click “Attack”

MANUAL TESTING WITH ZAP:

1. Browse your app normally through Firefox
2. ZAP captures all requests in “Sites” tab
3. Right-click request → Attack → Active Scan
4. Review alerts for vulnerabilities found

TESTING SPECIFIC ENDPOINTS:

1. Find request in ZAP (e.g., POST /api/login)
2. Right-click → “Open/Resend with Request Editor”
3. Modify parameters (try SQL injection payloads)
4. Click “Send”
5. Check response

8. USEFUL TOOLS

DATABASE VIEWERS:

SQLite Browser (View/Edit database.db): • Download: <https://sqlitebrowser.org/> • Open database.db file • Browse tables, run queries

DB Browser (Alternative): • VS Code extension: “SQLite Viewer”

CODE EDITORS:

VS Code (Recommended): • Download: <https://code.visualstudio.com/> • Extensions to install: - Python - ESLint - Prettier - SQLite Viewer

BROWSER DEVTOOLS:

Essential tabs: • Console: See JavaScript errors, test fetch commands • Network: See API requests/responses • Application: View localStorage, cookies • Elements: Inspect HTML/CSS

Quick access: Press F12

9. BEFORE SUBMISSION CHECKLIST

FUNCTIONALITY:

☐ Backend starts without errors: python app.py ☐ Frontend starts without errors: npm run dev ☐ All features work as intended ☐ Database creates successfully ☐ API endpoints return expected responses

VULNERABILITIES:

☐ Implemented 6+ vulnerabilities ☐ Tested each vulnerability works ☐ SQL injection actually bypasses auth ☐ XSS alert actually pops up ☐ IDOR actually shows other users' data ☐ Documented location of each vulnerability

TESTING:

☐ Fresh install works on clean machine ☐ README.md setup instructions are accurate ☐ Default credentials work (admin/admin123 or documented) ☐ Dependencies install successfully ☐ No hardcoded file paths (use relative paths) ☐ Venv instructions work (tested by teammate)

DOCUMENTATION:

☐ README.md includes: - Virtual environment setup - How to run backend (with venv activation) - How to run frontend - Default login credentials - List of features - List of vulnerabilities

☐ PROJECT_SUMMARY.pdf includes: - Team name and members - Application description - Features with screenshots - All 6+ vulnerabilities documented - Location, description, exploit for each

SUBMISSION:

☐ All files in one ZIP ☐ Named correctly: TeamName_Milestone1.zip ☐ No database.db included (it auto-creates) ☐ No node_modules/ folder (too large!) ☐ No venv/ folder (recreate with pip install) ☐ No **pycache/** folder ☐ .gitignore included ☐ All team member names in submission comments

GETTING HELP

IN CLASS: • Tuesday & Thursday: Full work sessions with instructor

OFFICE HOURS: • Tuesday/Thursday: 3:00-3:55 PM • Location: TEC 347

EMAIL: • oluseyi.olukola@usm.edu • Response time: Within 24 hours (weekdays)

CANVAS: • Check Files section for all resources • Discussion board for team communication

AI ASSISTANCE: • Use ChatGPT, Claude for code generation • Refer to AI Prompt Library for pre-written prompts • Always test AI-generated code!

QUICK REFERENCE - IMPORTANT URLS

Your Application: • Frontend: <http://localhost:3000> • Backend API: <http://localhost:5000> • Test endpoint: <http://localhost:5000/api/test>

Documentation: • Next.js: <https://nextjs.org/docs> • Flask: <https://flask.palletsprojects.com/> • SQLite: <https://www.sqlite.org/docs.html> • React: <https://react.dev/> • Python venv: <https://docs.python.org/3/library/venv.html>

Tools: • ZAP Proxy: <https://www.zaproxy.org/> • Postman: <https://www.postman.com/> • SQLite Browser: <https://sqlitebrowser.org/> • VS Code: <https://code.visualstudio.com/>

Canvas Resources: • Boilerplate template • AI Prompt Library • Vulnerability Implementation Guide • This Quick Reference

COMMON ERROR MESSAGES & FIXES

“externally-managed-environment” → Create and activate venv first!

“Cannot find module ‘X’” → npm install (frontend) or pip install X (backend, in venv)

“Port already in use” → kill -9 [PID] or use different port

“CORS error” → Add CORS(app) in Flask backend

“Database locked” → Close all connections, restart Flask

“Failed to fetch” → Check Flask is running on port 5000 with venv activated

“Unexpected token” → Check JSON syntax, commas, brackets

“404 Not Found” → Check URL, endpoint name, HTTP method

“500 Internal Server Error” → Check Flask terminal for error details

“(venv) not showing” → Run source venv/bin/activate (or venvon Windows)

KEYBOARD SHORTCUTS

VS Code: • Ctrl+(or Cmd+): Open terminal • Ctrl+P: Quick file open • Ctrl+Shift+P: Command palette • Ctrl+/: Toggle comment

Browser DevTools: • F12: Open DevTools • Ctrl+Shift+C: Inspect element • Ctrl+Shift+R: Hard refresh (clear cache)

Terminal: • Ctrl+C: Stop running process • Ctrl+L: Clear terminal • Tab: Auto-complete • Up arrow: Previous command

VIRTUAL ENVIRONMENT QUICK REFERENCE

CREATE (first time only):

```
python3 -m venv venv    # macOS/Linux
```

```
python -m venv venv     # Windows
```

ACTIVATE (every time before working):

```
source venv/bin/activate    # macOS/Linux
```

```
venv\Scripts\activate      # Windows
```

VERIFY ACTIVATION: • Look for (venv) in terminal prompt • Run: which python (should show venv path)

INSTALL PACKAGES (after activating):

```
pip install -r requirements.txt
```

DEACTIVATE (when done working):

```
deactivate
```

DELETE (if corrupted):

```
rm -rf venv            # macOS/Linux
```

```
rmdir /s venv         # Windows
```

Then recreate with: `python3 -m venv venv`

FINAL TIPS

✓ ALWAYS activate venv before pip install ✓ Save often, commit to git regularly ✓ Test after every change ✓ Read error messages carefully ✓ Google error messages if stuck ✓ Ask AI for help debugging ✓ Ask your team for help ✓ Don't wait until last minute ✓ Keep it simple - working > perfect ✓ Document as you go ✓ Have fun building!

Good luck with your capstone project! 🚀
